Conception des Bases de Données Relationnelles

Préparé par: Mme GHACHEM Amira
Public cible: 1ère année Ingénieur

Chapitre V

Le langage SQL

2ème partie:

Langage de Définition de Données (LDD)

Introduction

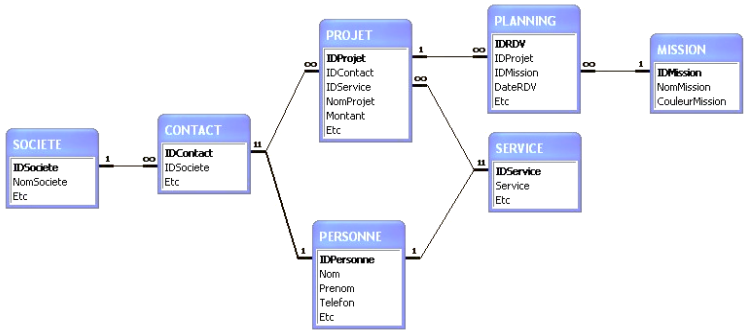
- *Le Langage de Définition de Données permet de:*
  - *Créer*
  - *Supprimer des objets dans la base de données*
    - *Ces données peuvent être:*
      - *Tables,*
      - *Contraintes d'intégrité (Clé primaire, clé étrangère, …),*
      - *Vues (copie personnalisée sur une ou plusieurs tables).*

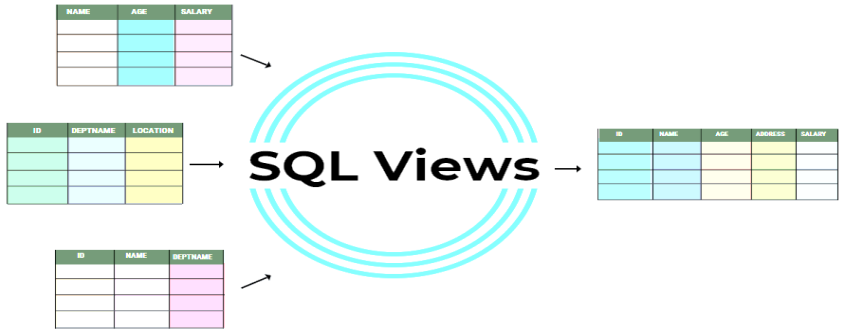




Introduction

- *Le LDD permet de créer les objets qui composent la base de données de façon déclarative.*
- *Il permet notamment:*
  - *La définition des schémas des relations.*
  - *La définition des contraintes d'intégrité.*
  - *La définition de vues relationnelles.*





Création de tables

- *La création d'une table est le fondement de la création d'une base de données en SQL.*
- *La création d'une table est la définition de:*
  - *un schéma de relation en intension,*
  - *tous les attributs le composant avec leurs domaines respectifs.*

Syntaxe

CREATE TABLE *<nom de la table>* ***(***
<nom colonne1><type colonne1>, *<nom colonne2><type colonne2>*,
*…,<nom colonneN><type colonneN>****);***

Création de tables

- **Exemple:**

CREATE TABLE *Personne* ***(***

*Nom VARCHAR(25), Prénom VARCHAR(25), Age INTEGER(3)****);***



Personne

Nom	Prénom	Age



Personne

Column Name	Data Type	Nullable	Default	Primary Key
Nom	VARCHAR2(25)	Yes	-	-
Prenom	VARCHAR2(25)	Yes	-	-
Age	NUMBER(3,0)	Yes	-	-

Affichage selon le SGBD Oracle

Type de données

- *Un attribut d'une table est défini pour un domaine.*
- *On peut également dire qu'il est d'un type particulier.*
- *Les types de données en SQL varient d'un SGBD à l'autre.*
- *Parmi les types standards que l'on retrouve dans tous les SGBD, on site:*
  - *Les types numériques,*
  - *Les types chaîne de caractères,*
  - *Les types date,*
  - *Les types énumérés,…*

Les types Numériques (1/3)

- *Les nombres entiers: INTEGER(X), où X est optionnel et désigne le nombre de chiffres maximum pouvant composer le nombre.*
- *Il existe également un certain nombre de variantes permettant de définir des entiers plus ou moins volumineux, tels que TINYINT, SMALLINT ou LONGINT.*

Les types Numériques (2/3)

- *Les nombres décimaux: DECIMAL(X,Y), où X et Y sont optionnels et désignent respectivement le nombre de chiffres maximum pouvant composer le nombre avant et après la virgule.*
- *NUMERIC est également utilisé de façon équivalente.*

Les types Numériques (3/3)

- *Les nombres à virgule flottante: REAL(X,Y) avec X et Y optionnels et définissent le nombre de chiffres avant et après la virgule.*
- *Il existe également un certain nombre de variantes permettant de définir une précision plus grande, telles que DOUBLE.*

Les types chaîne de caractères

- *On distingue principalement les types CHAR(X) et VARCHAR(X), où X est obligatoire et désigne la longueur de la chaîne.*
- *CHAR définit des chaînes de longueur fixe (complétée à droite par des espaces, si la longueur est inférieure à X).*
- *VARCHAR définit des chaînes de longueur variables.*
- *CHAR et VARCHAR sont généralement limités à 255 caractères.*
- *La plupart des SGBD proposent des types, tels que TEXT ou CLOB (Character Long Object), pour représenter des chaînes de caractères longues, jusqu'à 65000 caractères par exemple.*

Les types Date

- *Les types DATE dont introduit avec la norme SQL2, on distingue:*
  - *Le type DATE qui représente une date selon un format de type « AAAA-MM-JJ ».*
  - *Le type DATETIME qui représente une date avec un horaire, dans un format tel que « AAAA-MM-JJ HH:MM:SS ».*
  - *Il existe également des restrictions de ces types, tels que YEAR, TIME, etc.*

Autres types

- *En fonction du SGBD, il peut exister de nombreux autres types. On peut citer par exemple:*
  - *Les types monétaires pour représenter des décimaux associés à une monnaie,*
  - *Les types ENUM pour représenter des énumérations,*
  - *Les types BLOB (Binary Long Object) pour représenter des données binaires tels que des documents multimédia (images bitmap, vidéo, etc).*

Le type absence de valeur

- *L'absence de valeur, représentée par la valeur **NULL** est une information fondamentale en SQL.*
- *Il ne faut pas la confondre avec la chaîne espace de caractères où bien la valeur 0.*
- *Il ne s 'agit pas d'un type à proprement parlé, mais d'une valeur possible dans tous les types.*

Définition des contraintes d'intégrité

- *La définition des attributs n'est pas suffisante pour définir un schéma relationnel, il faut lui adjoindre la définition de contraintes d'intégrité, qui permettent de poser les notions de clé, d'intégrité référentielle, de restriction de domaine, etc.*
- *Une contrainte d'intégrité est une règle qui définit la cohérence d'une donnée ou d'un ensemble de données de la base de données.*
- *Les contraintes sont définies au moment de la création des tables.*
- *Il existe deux types de contraintes:*
  - *Sur une colonne unique,*
  - *Ou sur une table lorsque la contrainte porte sur une ou plusieurs colonnes.*

Contrainte d'intégrité sur une colonne

- ***PRIMARY KEY:*** *définit l'attribut comme clé primaire.*
- ***NOT NULL :*** interdit l'absence de valeur pour l'attribut.
- ***UNIQUE:*** que deux tuples de la table aient la même valeur pour l'attribut.
- ***REFERENCES <nom table> (<nom colonne>):*** contrôle l'intégrité référentielle entre l'attribut et la table et ses colonnes spécifiées.
- ***CHECK (<Condition>):*** contrôle la validité de la valeur de l'attribut spécifié dans la condition dans le cadre d'une restriction de domaine.

Contrainte d'intégrité sur une table

- **PRIMARY KEY (<liste d'attributs>):** *définit les attributs de la liste comme la clé primaire.*
- **UNIQUE (<liste d'attributs>):** *interdit que deux tuples de la relation aient les mêmes valeurs pour l'ensemble des attributs de la liste.*
- **FOREIGN KEY (<liste d'attributs>) RREFRENCES <nom table> (>nom colonne>):** *contrôle l'intégrité référentielle entre les attributs de la liste et la table et ses colonnes spécifiées.*
- **CHECK (<condition>):** *contrôle la validité de la valeur des attributs spécifiées dans la condition dans le cadre d'une restriction de domaine.*

Contrainte d'intégrité

- ***Syntaxe:***

  CREATE TABLE *<nom de la table>* ***(***

  *<nom colonne 1> <type colonne 1> [<**contrainte** colonne 1>],*

  *<nom colonne 2> <type colonne 2> [<**contrainte** colonne 2>],*

  …

  *<nom colonne N> <type colonne N> [<**contrainte** colonne N>],*

  <contraintes de table>

  );

Contrainte d'intégrité (Exemple)

- ***Exemple:***

```
CREATE TABLE  Personne (
 N°CIN CHAR(13) PRIMARY KEY,
 Nom VARCHAR(25) NOT NULL,
 Prenom VARCHAR(25) NOT NULL,
 Age INTEGER(3) CHECK (Age BETWEEN 18 AND 65),
 Mariage CHAR(13) REFERENCES Personne(N°CIN),
 Codepostal INTEGER(5),
 Pays VARCHAR(50)  UNIQUE (Nom, Prenom),
 FOREIGN KEY (Codepostal, Pays) REFERENCES Adresse (CP,Pays)
 );
 CREATE TABLE Adresse (
 CP INTEGER(5) NOT NULL,
 Pays VARCHAR(50) NOT NULL,
 initiale CHAR(1) CHECK (initiale =LEFT(Pays,1)),
 PRIMARY KEY (CP, Pays)
 );
```

Exemple (suite)

- *Dans la définition du schéma précédent on a posé les contraintes suivantes:*
  - *La clé primaire de la table Personne est N°CIN et la clé primaire de la table Adresse est (CP, Pays).*
  - *Nom, Prenom ne peuvent pas être NULL et (Nom, Prenom) est une clé.*
  - *Age doit être compris entre 18 et 65 ans et Initiale doit être la première lettre de Pays (avec la fonction LEFT qui renvoie la sous chaîne à gauche de la chaîne passée en premier argument, sur le nombre de caractères passées en second argument).*
  - *Mariage est une clé étrangère vers la table Personne et (Codepostal, Pays) est une clé étrangère vers la table Adresse.*

Création de Vues (1/3)

- *Une vue est une définition logique d'une table, sans stockage de données, obtenue par interrogation d'une ou de plusieurs tables de la bases de données.*
- *Une vue peut donc être perçue comme une fenêtre dynamique sur les données, ou encore une requête stockée (mais dont seule la définition est stockée, pas le résultat, qui reste calculé dynamiquement).*

Création de Vues (2/3)

- *Une vue permet d'implémenter le concept de schéma externe d'un modèle conceptuel.*
- *Une vue permet de réutiliser le résultat d'une requête dans d'autres requêtes comme s'il s'agissait d'une nouvelle table*
  - *bien qu'elle soit régénérée dynamiquement lors de son utilisation.*

Création de Vues (3/3)

- ***Syntaxe:***

CREATE VIEW <nom de vue> [<nom des colonnes>] AS <spécification d'une requête>

- *La spécification d'une requête se fait en utilisant le Langage de Manipulation de Données LMD:*
  - *Select, insert into, delete, update,…*
- *Le nombre de colonnes nommées doit être égal au nombre de colonnes renvoyées par la question spécifiée.*
- *Le nom des colonnes est optionnel, s'il n'est pas spécifié, c'est le nom des colonnes telle qu'elles sont renvoyées par la requête, qui sera utilisé.*

Création de Vues (Exemple)

- *Nom des clients qui ont loué des voitures le mois de Mars 2019.*
  - *CREATE VIEW Client-Mars(NumCIN, Nom) AS*
    SELECT NumCIN, Nom
    *FROM CLIENT **C**, LOUER **L***
    WHERE Date >= '01/03/2019' AND Date<= '31/03/2019'
    *AND **C**.NumCIN= **L**.NumCIN;*
- *Utilisation de la Vue:*

Noms des clients qui ont loué des voitures le mois de Mars 2019 en affichant les immatriculations des voitures.

  - *SELECT Nom, Immat*
    *FROM Client-Mars **CM**, LOUER **L***
    *WHERE **CM**.NumCIN=**L**.NumCIN;*

Remarques sur les Vues (1/4)

- ***Remarque 1: Vue sur une vue***
  - *Une vue peut avoir comme source une autre vue.*
  - *Exemple: les clients dont les NumCIN >3456 qui ont loué des voitures le mois de Mars 2019.*
    - *CERATE VIEW Client-Mars-3456 AS*
      *SELECT **
      FROM Client-Mars
      WHERE NumCIN >3456;
- ***Remarque 2: Vue en lecture et vue en écriture***
  - *Une vue est toujours disponible en lecture, à condition que l'utilisateur ait les droits spécifiés grâce au LCD (Langage de Contrôle de Données)*
  - *Une vue peut également être disponible en écriture dans certains cas, que l'on peut restreindre aux cas où la requête ne porte que sur une seule table.*

Remarques sur les Vues (2/4)

- *Dans certains cas, il est possible de modifier une vue issue de plusieurs tables.*
- *Dans le cas où une vue est destinée à être utilisée pour modifier des données, il est possible d'ajouter la clause « WITH CHECK OPTION » après la spécification de requête, afin de préciser que les données modifiées ou ajoutées doivent effectivement appartenir à la vue.*
- **CREATE VIEW nom_vue [(nom_col1,...)] AS**

  SELECT ... WITH CHECK OPTION ;

Remarques sur les Vues (3/4)

- ***Exemple***:

```
CREATE VIEW emp10 AS
SELECT *
FROM emp
WHERE n_dept = 10 WITH CHECK OPTION;
```

- *La condition WITH CHECK OPTION permet de vérifier que la mise à jour ou l'insertion faite à travers la vue ne produisent que des lignes qui font partie de la sélection de la vue.*

- *Ainsi donc, si la vue emp10 a été créée avec WITH CHECK OPTION, on ne pourra à travers cette vue ni modifier, ni insérer des employés ne faisant pas partie du département 10.*

Remarques sur les vues (4/4)

- *Il est possible d'effectuer des INSERT et des UPDATE à travers des vues, sous deux conditions :*
  - *La clause SELECT définissant la vue ne doit pas comporter de jointure,*
  - *les colonnes résultat du SELECT doivent être des colonnes réelles et non pas des expressions.*

- ***Exemple** :*

  - *Modification des salaires du département 10 à travers la vue emp10.*
    - *On va faire une augmentation de salaire des employés du département 10 par 10%.*

```
UPDATE emp10 SET sal =(sal+(sal*0,1));
```

→Toutes les lignes de la table emp, telles que le contenu de la colonne n_dept est égal à 10 seront modifiées.

Suppression d'objets

- *Il est possible de supprimer des objets de la BD, tels que les tables ou les vues.*
- ***Syntaxe:***
  - ***DROP <Type objet> <nom objet>;***
- ***Exemple:***
  - *DROP table CLIENT;*
  - *DROP VIEW Client-Mars;*

Modification des tables (1/2)

- *L'instruction ALTER TABLE permet de modifier la définition d'une table (colonnes ou contraintes) préalablement créée.*
- ***Syntaxe: Ajout de colonne***
  - ***ALTER TABLE <nom de table>***
    ADD (<définition de colonnes>);
  - ***Exemple:***
    ALTER TABLE CLIENT
    ADD (Num-tel INTERGER(8));
- ***Syntaxe: Suppression de colonne***
  - ***ALTER TABLE <nom table>***
    DROP (<nom de colonne>);
  - ***Exemple:***
    ALTER TABLE VOITURE
    DROP (Marque);

Modification des tables (2/2)

- ***Syntaxe: Modification de colonne***
  - ***ALTER TABLE <nom de table>***
    MODIFY (<redéfinition de colonnes>);
  - ***Exemple:***
    ALTER TABLE Personne
    MODIFY (Nom VARCHAR(25));
- ***Syntaxe: Ajout de contrainte***
  - ***ALTER TABLE <nom de table>***
    ADD Constraint (<définition de contrainte de table>);
  - ***Exemple:***
    ALTER TABLE CLIENT
    ADD Constraint (UNIQUE(Nom));

Exercice (LDD)

- Soit le modèle relationnel suivant :

Producteur (**raison_sociale**:chaîne(25),ville:chaîne(255))
Consommateur (**login**:chaîne(10),**email**:chaîne(50), nom:chaîne(50),prenom:chaîne (50), ville:chaîne(255))
Produit (**id**:entier, description:chaîne(100), #produit-par, #consomme-par-login, #consomme-par-email)

On ajoute que :

- (nom,prenom,ville) est une clé candidate de Consommateur.
- Tous les produits sont produits.
- Tous les produits ne sont pas consommés.

1. Établir le code LDD standard permettant d'implémenter ce modèle en SQL.
2. Insérez les données dans votre base de données correspondant aux assertions suivantes :
   a. L'entreprise de **Compiègne** "**Pommes Picardes SARL**" a produit **4 lots de pommes**, et **2 lots de cidre**.
   b. Il existe trois utilisateurs consommateurs dans la base, donc les adresses mails sont :

   Al.Un@compiegne.fr , Bob.Deux@compiegne.fr , Charlie.Trois@compiegne.fr

   a. Ce sont des employés de la ville de **Compiègne** qui habitent cette ville.

   → Leur mail est construit sur le modèle **Prenom.Nom@compiegne.fr.**

   → Leur **login** est leur **prénom**
3. Modifiez les données de votre base de données pour intégrer les assertions suivantes :
   a. **1 lots de pommes a été consommés par Al.Un.**
   b. **2 lots de pomme ont été consommé par Bob.**
   c. **Tous les lots de cidre ont été consommés par Al.**

Correction:

- ***1. code SQL correspondant à la création des Tables Producteur, Consommateur et Produit:***

```
-- Table Producteur
CREATE TABLE Producteur (
raison_sociale VARCHAR (25),
ville VARCHAR(255),
PRIMARY KEY (raison_sociale)
);

-- Table Producteur
CREATE TABLE Consommateur (
login VARCHAR(10),
email VARCHAR(50),
nom VARCHAR(50) NOT NULL,
prenom VARCHAR(50) NOT NULL,
ville VARCHAR(255) NOT NULL,
PRIMARY KEY (login,email),
UNIQUE (nom,prenom,ville)
);
```

```
-- Table Producteur
CREATE TABLE Produit (
id INTEGER,
description VARCHAR(100),
produit_par VARCHAR(25) NOT NULL,
consomme_par_login VARCHAR(10),
consomme_par_email VARCHAR(50),
PRIMARY KEY (id),
FOREIGN KEY (produit_par) REFERENCES
Producteur(raison_sociale),
FOREIGN KEY
(consomme_par_login,consomme_par_emai
l) REFERENCES
Consommateur(login,email)
);
```

Correction:

- ***2. Insertion de données dans la base de données:***

```
-- Insertion du producteur
INSERT INTO Producteur (raison_sociale, ville)
VALUES ('Pommes Picardes SARL', 'Compiègne');

-- Insertion des produits
INSERT INTO Produit (id, description, produit_par)
VALUES (1, 'Lot de pommes', 'Pommes Picardes SARL');
INSERT INTO Produit (id, description, produit_par)
VALUES (2, 'Lot de pommes', 'Pommes Picardes SARL');
INSERT INTO Produit (id, description, produit_par)
VALUES (3, 'Lot de pommes', 'Pommes Picardes SARL');
INSERT INTO Produit (id, description, produit_par)
VALUES (4, 'Lot de pommes', 'Pommes Picardes SARL');
INSERT INTO Produit (id, description, produit_par)
VALUES (5, 'Lot de cidre', 'Pommes Picardes SARL');
INSERT INTO Produit (id, description, produit_par)
VALUES (6, 'Lot de cidre', 'Pommes Picardes SARL');

-- Insertion des consommateurs
INSERT INTO Consommateur (login, email, nom, prenom, ville)
VALUES ('Al', 'Al.Un@compiegne.fr', 'Un', 'Al', 'Compiègne');
INSERT INTO Consommateur (login, email, nom, prenom, ville)
VALUES ('Bob', 'Bob.Deux@compiegne.fr', 'Deux', 'Bob', 'Compiègne');
INSERT INTO Consommateur (login, email, nom, prenom, ville)
VALUES ('Charlie', 'Charlie.Trois@compiegne.fr', 'Trois', 'Charlie',
'Compiègne');
```

Correction:

- ***3.Mise à jours des tables:***

--Mise à jours de la table Produit

```
UPDATE produit
SET consomme_par_login='Al',
consomme_par_email='Al.Un@compiegne.fr'
WHERE id=1;
UPDATE produit
SET consomme_par_login='Bob',
consomme_par_email='Bob.Deux@compiegne.fr'
WHERE id=2 OR id=3;
UPDATE produit
SET consomme_par_login='Al',
consomme_par_email='Al.Un@compiegne.fr'
WHERE description='Lot de cidre';
```

3ème partie:

Langage de Contrôle de Données (LCD)

Introduction

- *Le LCD permet de créer les utilisateurs et de définir leurs droits sur les objets de la BD de façon déclarative.*
- *Il permet notamment l'attribution et la révocation de droits:*
  - *à des utilisateurs,*
  - *sur l'ensemble des bases du SGBD,*
  - *sur une BD en particulier,*
  - *sur des relations d'une BD,*
  - *sur certains attributs seulement d'une relation.*

Attribution de droits (1/2)

- *SQL propose une commande pour attribuer des droits à des utilisateurs sur des tables.*
- ***Syntaxe:***
  - ***GRANT <liste de droits> ON <nom table> TO <utilisateur> [WITH GRANT OPTION];***
- *Les droits disponibles renvoient directement aux instructions SQL que l'utilisateur peut exécuter :*
  - ***SELECT***
  - ***INSERT***
  - ***DELETE***
  - ***UPDATE***
  - ***ALTER, DROP, CREATE***
- *De plus il est possible de spécifier le droit* ***ALL PRIVILEGES*** *qui donne tous les droits à l'utilisateur (sauf celui de transmettre ses droits).*

Attribution de droits (2/2)

- ***Exemple:***
  - *GRANT SELECT, UPDATE ON Personne TO Ahmed;*
  - *GRANT ALL PRIVILEGES ON Adresse TO PUBLIC;*
- *La clause **WITH GRANT OPTION** est optionnelle, elle permet de préciser que l'utilisateur a le droit de transférer ses propres droits sur la table à d'autres utilisateur.*
- *Une telle clause permet une gestion décentralisée de l'attribution des droits et non reposant uniquement dans les mains d'un administrateur unique.*
- *La spécification **PUBLIC** à la place d'un nom d'utilisateur permet de donner les droits spécifiés à tous les utilisateurs de la BD.*

Attribution de droits (Remarques)

- ***Remarque 1: Droits sur un SGBD***
  - *Certains SGBD permettent de spécifier des droits au niveau du SGBD, c'est-à-dire pour toutes les tables de toutes les BDs du SGBD.*
  - *La syntaxe dans le cas de MySQL est « . » à la place du nom de la table.*
  - *Dans ce cas les droits* ***CREATE*** *et* ***DROP*** *sont généralement ajoutés pour permettre ou non aux utilisateurs de créer et supprimer des BD et des tables.*

Attribution de droits (Remarques)

- ***Remarque 2: Droits sur une BD***
  - *Certains SGBD permettent de spécifier des droits au niveau d'une BD, c'est-à-dire pour toutes les tables de cette base de données.*
  - *La syntaxe dans ce cas de MySQL est « nom_bd » à la place de nom de la table.*
  - *Dans ce cas les droits CREATE et DROP sont généralement ajoutés pour permettre ou non aux utilisateurs de créer et supprimer des tables sur cette BD.*

Attribution de droits (Remarques)

- ***Remarque 3: Droits sur une Vue***
  - *Il est possible de spécifier des droits sur des vues plutôt que sur des tables, avec une syntaxe identique (et un nom de vue à la place d'un nom de table).*
- ***Remarque 4: Catalogue de données***
  - *Les droits sont stockés dans le catalogue des données, il est généralement possibles de modifier directement ce catalogue à la place d'utiliser la commande* ***GRANT*** *(cela reste néanmoins déconseillé).*

Attribution de droits (Remarques)

- ***Remarque 5: création des utilisateurs***
  - *Les modalités de création d'utilisateurs, voire de groupes d'utilisateurs dans le SGBD, reste dépendantes de celui-ci.*
  - *Des commande SQL peuvent être disponibles, telles que **CREATE USER**, ou bien la commande **GRANT** lorsque qu'elle porte sur un utilisateur non existant peut être chargée de créer cet utilisateur.*
    - *ajouter dans cette base de données un nouvel utilisateur identifié par « Pers10 » et ayant le mot de passe « 2tSm66yx »:*
      - **CREATE USER Pers10 IDENTIFIED BY 2tSm66yx ;**
    - *attribuer à l'utilisateur « Pers10 », les droits de sélection, de modification et de suppression de données sur la table « REALISATION ».*
      - **GRANT SELECT, UPDATE, DELETE ON REALISATION TO Pers10;**
  - *Des modules spécifiques d'administration sont généralement disponibles pour prendre en charge la gestion des utilisateurs.*

Révocation de droits

- *SQL propose une commande pour révoquer les droits attribués à des utilisateurs.*
- ***Syntaxe:***
  - ***REVOKE <liste de droits> ON <nom table>***
    FROM <utilisateurs>;
- ***Exemple:***
  - *REVOKE SELECT, UPDATE ON Personne FROM Ahmed;*
  - *REVOKE ALL PRIVILEGES ON Adresse FROM PUBLIC;*

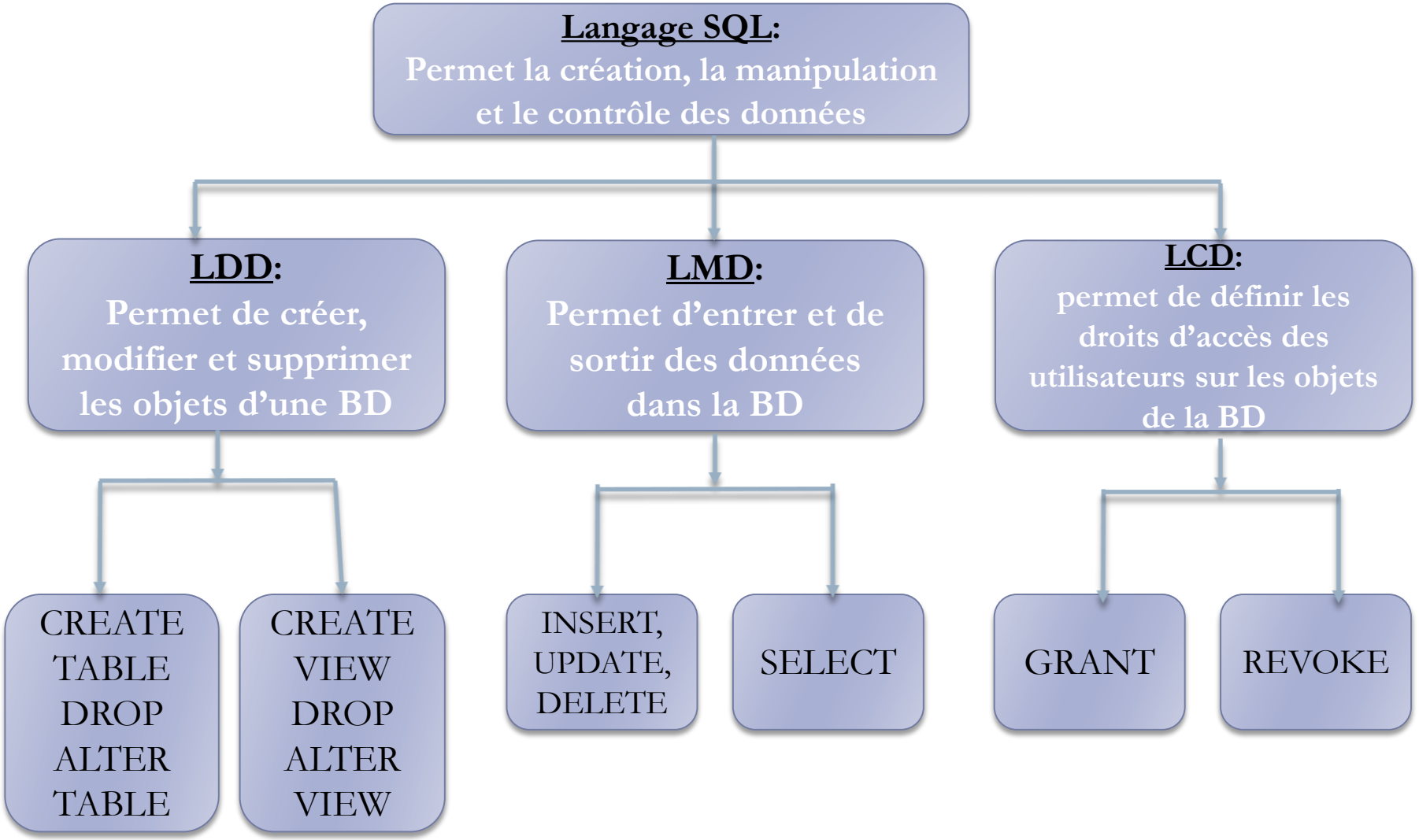
Révocation de droits (Remarques)

- ***Remarque 1: Révocation du droit d'attribution des droits***
  - *Pour retirer les droits de donner les droits à un utilisateur (qui l'a donc obtenu par la clause WITH GRANT OPTION),*
    - *il faut donc utiliser la valeur GRANT OPTION dans la liste des droits révoqués.*
    - ***Syntaxe**:*

      REVOKE GRANT OPTION FOR <liste de droits>

      ON object FROM <utilisateurs>;
- ***Remarque 2: Révocation en cascade***
  - *Lorsque qu'un droit est supprimé pour un utilisateur, il l'est également pour tous les utilisateurs qui avait obtenu ce même droit par l'utilisateur en question.*

Synthèse


Langage SQL:
Permet la création, la manipulation et le contrôle des données
LDD:
Permet de créer, modifier et supprimer les objets d'une BD
LMD:
Permet d'entrer et de sortir des données dans la BD
LCD:
permet de définir les droits d'accès des utilisateurs sur les objets de la BD
CREATE TABLE DROP ALTER TABLE
CREATE VIEW DROP ALTER VIEW
INSERT, UPDATE, DELETE
SELECT
GRANT
REVOKE

Exercice (LCD)

- Soit la base de données suivantes :
  - **SPECTACLE** (**nospectacle**, nom, durée, type)
  - **REPRESENTATION** (**date**, #**nospectacle**, #**nosalle**, prix)
  - **SALLE** (**nosalle**, nbplaces)
- On suppose des classes d'utilisateurs qui ont accès à tout ou partie de ce schéma relationnel :
  1. Le programmateur qui entre les spectacles dans la base de données,
  2. Le régisseur qui gère les salles et les représentations,
  3. Les clients qui peuvent accéder au programme.
- Donner les droits associés à chaque classe d'utilisateurs.

Correction

1. GRANT INSERT, UPDATE, DELETE ON SPECTACLE TO programmateur;

2. GRANT INSERT, UPDATE, DELETE ON SALLE, REPRESENTATION TO régisseur;

3. GRANT SELECT ON SPECTACLE, SALLE, REPRESENTATION TO PUBLIC;